

ORM = Object Relational Mapping

Date |

Page |

#FSD Unit-3 Answers (QB)

Section A. (2M)

1. Role of indexing in database optimization
 - (i) Speeds up data retrieval
 - (ii) Reduces full table scans
 - (iii) Uses structure like B-tree/hash
 - (iv) Improves query performance
2. Purpose of ORM in app development
 - (i) Maps tables to objects
 - (ii) Reduces SQL writing
 - (iii) Improves code readability
 - (iv) Ensures database abstraction
3. Common server side validation techniques
 - (i) Type validation
 - (ii) Length checks
 - (iii) Regex pattern matching
 - (iv) Null/empty check
4. Security measures for form validation
 - (i) Input sanitization
 - (ii) Use prepared statements
 - (iii) CSRF protection
 - (iv) CAPTCHA usage
5. Primary goals of unit testing
 - (i) Test individual units
 - (ii) Detect bugs early
 - (iii) Ensure code correctness
 - (iv) Support refactoring

CSRF = Cross
Site
Request
Forgery

SSL = Secure Sockets Layer
TLS = Transport Layer Security

Date |

Page |

6. Advantages of API testing

- (i) Faster than UI testing
- (ii) Independent of front end
- (iii) Early automation
- (iv) Improves backend reliability

7. Manual vs Automated testing

- (i) Manual: human effort
- (ii) Automated: tool based
- (iii) Manual is slow
- (iv) Automated is repeatable

8. ORM tools used in web apps

- (i) Hibernate
- (ii) Sequelize
- (iii) Entity Framework
- (iv) Django ORM

9. Validation rules with data types

- (i) Email $\rightarrow$ regex
- (ii) Number $\rightarrow$ range check
- (iii) String $\rightarrow$ length check
- (iv) Date $\rightarrow$ format check

10. Role of HTTPS in server security

- (i) Encrypts data (SSL/TLS)
- (ii) Prevents data interception
- (iii) Ensures data integrity
- (iv) Authenticates server

11. Unit testing vs API testing

- (i) Unit: tests small code parts
- (ii) API: tests endpoints
- (iii) Unit: developer focused
- (iv) API: integration focused

12. Basic CI/CD terminology

- (i) CI = Code Integration
- (ii) CD = Deployment process
- (iii) Pipeline: automated steps
- (iv) Build: code compilation

13. Cloud deployment models

- (i) Public ~~model~~ cloud
- (ii) Private ~~no~~ cloud
- (iii) Hybrid cloud
- (iv) Community cloud

XSS =
Cross
Site
Scripting

IaaS = Infrastructure as a Service
PaaS = Platform as a Service
FaaS = Function as a Service

SoaS = Software
as a
Service

Date |

Page |

14. Objectives of CI/CD pipelines

- (i) Automate builds
- (ii) Run tests automatically
- (iii) Faster delivery
- (iv) Reduce human error

15. SQL query optimization techniques

- (i) Use indexing
- (ii) Avoid SELECT *
- (iii) Optimize joins
- (iv) Normalize tables

16. Primary ORM mapping types

- (i) One to One
- (ii) One to Many
- (iii) Many to One
- (iv) Many to Many

17. Authentication vs Authorization

- (i) Authentication = identify check
- (ii) Authorization = access control
- (iii) Auth uses login credentials
- (iv) Authorization uses roles/permissions

18. Cloud service models

- (i) IaaS
- (ii) PaaS
- (iii) SaaS
- (iv) FaaS

19. Test case structure

- (i) Test ID
- (ii) Input data
- (iii) Expected result
- (iv) Actual result

20. Common API testing tools

- (i) Postman
- (ii) Soap UI
- (iii) JMeter
- (iv) Rest Assured

21. Deployment workflow stages

- (i) Build
- (ii) Test
- (iii) Deploy
- (iv) Monitor

22. Benefits of DB Normalization

- (i) Reduce redundancy
- (ii) Improves consistency
- (iii) Saves storage
- (iv) Enhances integrity

23. Security threats in server

- (i) SQL injection
- (ii) XSS
- (iii) CSRF
- (iv) DDoS

24. CI/CD pipeline components

- (i) Source control (Git)
- (ii) Build tools
- (iii) Testing tools
- (iv) Deployment Tools

Section B (5M)

1. Apply ORM techniques to improve DB interaction efficiency
 - (i) Use ORM tools like Hibernate or Sequelize
 - (ii) Map DB tables to classes/objects
 - (iii) Reduce manual SQL query writing
 - (iv) Use lazy loading to fetch required data only
 - (v) Implement caching for faster access
 - (vi) Maintain relationships using ORM mappings
 - (vii) Improve code maintainability & portability
2. Demonstrate server side validation for user registration forms
 - (i) Validate username & password length
 - (ii) Check email format using regex
 - (iii) Ensure required fields are not empty
 - (iv) Sanitize inputs to prevent SQL injection
 - (v) Use password hashing before storage
 - (vi) Verify uniqueness of username/email
 - (vii) Return proper validation error messages
3. Implement unit testing for a login module
 - (i) Test valid username & password ~~length~~ inputs
 - (ii) Verify invalid login credentials handling
 - (iii) Check empty input field conditions
 - (iv) Test password encryption logic
 - (v) Use password like JUnit or PyTest
 - (vi) Compare expected & actual outputs
 - (vii) ~~Compare expected & actual outputs~~
 - (viii) Automate repeated test execution

4. Apply API testing ~~strategy~~ strategies to verify REST endpoints

- (i) Test GET, POST, PUT, DELETE methods
- (ii) Verify response status codes
- (iii) Validate JSON/XML response data
- (iv) Check authentication vs authorization
- (v) Perform boundary & negative testing
- (vi) Automate testing using Postman
- (vii) Measure API response time & reliability

5. Analyze security vulnerabilities in server side apps

- (i) SQL injection through ~~un~~ unsanitized queries
- (ii) Cross Site Scripting (XSS) attacks
- (iii) Cross Site Request Forgery (CSRF) risks
- (iv) Weak password storage mechanisms
- (v) Insecure session management
- (vi) Lack of input validation
- (vii) Misconfigured server permissions

6. Examine the impact of indexing on database performance

- (i) Speeds up data retrieval operations
- (ii) Reduced table scanning time
- (iii) Improves WHERE clause performance
- (iv) Enhances JOIN operation efficiency
- (v) Increased performance of sorting/searching
- (vi) Excessive indexing increases storage usage
- (vii) Too many ~~use~~ indexing indexes slow INSERT/UPDATE operations

7. Demonstrate cloud deployment steps for a web app
- (i) Prepare application source code
 - (ii) Choose a cloud platform like AWS or Azure
 - (iii) Configure server and DB services
 - (iv) Upload application files to cloud server
 - (v) Set environment variables & dependencies
 - (vi) Perform deployment testing
 - (vii) Monitor application performance after deployment
8. Analyze CI/CD workflow efficiency in software delivery
- (i) Automate build & deployment processes
 - (ii) Reduces manual intervention
 - (iii) Detects bugs early through automated testing
 - (iv) Enables faster software releases
 - (v) Improves collaboration among developers
 - (vi) Ensures continuous integration of code changes
 - (vii) Enhances software reliability & consistency
9. Apply database optimization techniques to reduce query time
- (i) Create indexes on frequently searched columns
 - (ii) Normalize tables to reduce redundancy
 - (iii) Optimize SQL queries & joins
 - (iv) Avoid unnecessary SELECT * statements
 - (v) Use caching for repeated queries
 - (vi) Partition large tables when required
 - (vii) Monitor query execution plans regularly

10. Examine ORM limitations in large scale systems

- (i) Can generate inefficient SQL queries
- (ii) Performance overhead in complex operations
- (iii) Limited control over optimized SQL
- (iv) Difficulties handling large datasets
- (v) Increased memory consumption
- (vi) ORM abstraction may hide DB behavior
- (vii) Complex debugging in large applications

11. Implement API test automation using a testing framework

- (i) Select frameworks like Postman or Rest assured
- (ii) Create automated test scripts
- (iii) Send API requests programmatically
- (iv) Validate response status & ~~code~~ body
- (v) Use assertions for expected outputs
- (vi) Run tests repeatedly through CI/CD pipelines
- (vii) Generate automated test reports

12. Analyze server side security practices for data protection

- (i) Use HTTPS for secure communication
- (ii) Encrypt sensitive user data
- (iii) Implement authentication & authorization
- (iv) Apply input validation & sanitization
- (v) Use firewalls & intrusion detection systems
- (vi) Perform regular security updates & patches
- (vii) Maintain backup & recovery mechanisms

Section C (10M)

1. Analyze cloud deployment challenges & mitigation strategies

(a) Cloud deployment challenges —

- (i) Security risks: data breaches, unauthorized access and insecure APIs can affect applications
- (ii) Downtime issues: server failures or network outages may reduce availability
- (iii) Scalability problems: sudden traffic increase can overload resources
- (iv) Vendor Lock-in: dependency on one cloud provider limits flexibility
- (v) Data Migration complexity: moving large datasets to cloud env. is difficult
- (vi) Compliance issues: organizations must follow regulations like GDPR or HIPAA
- (vii) Cost management: poor resource allocation may increase cloud expenses

(b) Mitigation strategies —

- (i) Implement encryption & multi factor authentication
- (ii) Use load balancing & backup servers for availability
- (iii) Enable auto scaling to handle varying traffic loads
- (iv) Adopt containerization & multi cloud strategies
- (v) Perform phased migration with proper testing
- (vi) Monitor compliance using auditing tool
- (vii) Use cost monitoring & resource organization techniques

<< Can also add points explaining cloud technology >>

2. Evaluate ORM performance compared to native SQL Querying

(a) ORM Performance -

(i) ORM tools like Hibernate simplify database interaction through object mapping

(ii) Reduce development time by avoiding manual SQL writing

(iii) Improve maintainability & portability of code

(iv) Provide automatic relationship handling & caching

(b) Native SQL Performance -

(i) Native SQL offers direct database control

(ii) Faster execution for complex joins & bulk operations

(iii) Allows query level optimization by developers

(iv) Better suited for high performance enterprise systems

(c) Comparison -

(i) ORM is easier for rapid development

(ii) Native SQL performs better in complex queries

(iii) ORM may generate inefficient queries in large systems

(iv) Native SQL requires more coding effort & database expertise

(v) ORM is preferred for maintainability; SQL for maximum performance

<< Also, explain ORM & SQL
along with >>

3. Design a secure CI/CD pipeline for cloud based application

(a) Steps in secure CI/CD pipeline design —

(i) Version control security:

Use Git repositories with access control & branch protection

(ii) Automated build process:

Compile and package code automatically after commits

(iii) ~~Security scanning~~ Automated testing:

~~Use vuln~~ Perform unit, integration, and API testing automatically

(iv) Security scanning:

Use vulnerability scanners and static code analysis tools

(vi) Secure deployment:

Deploy using HTTPS, SSH Keys, and encrypted secrets

(vii) Monitoring and Logging:

Monitor application logs and detect suspicious activities

(b) Benefits —

(i) Faster software delivery

(ii) Reduced human errors

(iii) Improved application security

(iv) Continuous monitoring & reliability

<< Can explain concepts
of CI/CD & cloud applications >>

4. Evaluate testing strategies for enterprise level applications

(a) Testing Strategies -

- (i) Unit testing: tests individual modules independently
- (ii) Integration testing: verifies individuals between modules
- (iii) System testing: tests the complete application system
- (iv) API testing: ensures backend services work correctly
- (v) Performance testing: measures speed, scalability, and stability
- (vi) Security testing: detects vulnerabilities and threats
- (vii) User Acceptance testing (UAT): confirms business requirements are met

(b) Evaluation -

- (i) Automated testing improves efficiency and coverage
- (ii) Manual testing helps identify usability issues
- (iii) Continuous testing supports DevOps workflows
- (iv) Enterprise systems require scalability and reliability checks
- (v) Combining multiple testing strategies ensures better software quality

<< Can give benefits & applications of testing >>

5. Design an optimized database architecture using ORM principles

(a) ORM based database architecture -

(i) Use ORM frameworks like Hibernate for object table mapping

(ii) Design normalized tables to reduce redundancy

(iii) Define relationships using One to One, One to Many, Many to One, and Many to Many mappings

(iv) Use indexing on frequently accessed columns

(v) Implement lazy loading for efficient memory usage

(vi) Apply caching mechanisms to reduce database queries

(vii) Use connection pooling for efficient resource utilization

(b) Optimization benefits -

(i) Faster data access & retrieval

(ii) Reduced development complexity

(iii) Better maintainability of applications

(iv) Improved scalability

(v) Better performance

<< Explain ORM (theory)

and construct a simple

diagram for the architecture >>

6. Analyze CI/CD implementation benefits in DevOps environments

(a) Benefits of CI/CD in DevOps —

(i) Automates software build, testing, and deployment processes

(ii) Reduces manual intervention and human errors

(iii) Enables frequent & faster software releases

(iv) Detects bugs early through continuous testing

(v) Improves collaboration between development and operations teams

(vi) Ensures consistent deployment across environments

(vii) Enhances application reliability and customer satisfaction

(b) Impact on DevOps —

(i) Supports agile development practices

(ii) Increases deployment efficiency & productivity

(iii) Shortens software delivery lifecycle

(iv) Provides rapid feedback & continuous improvements

<< Explain CI/CD concepts
alongside explaining what
is DevOps & >>